

Lab: Code Verification and Z3 Theorem Prover

(Week 5)

Yulei Sui

School of Computer Science and Engineering

University of New South Wales, Australia

Assignment-1 Marks Released

Marks are out (please check via <https://cgi.cse.unsw.edu.au/~give/Student/sturec.php>) and let us go through some Assignment-1 issues!

Assignment-2 Spec and Code Released

Remember to `git pull` or `docker pull`!

Quiz-2, Exercise-2 and Assignment-2

- Quiz-2 with 25 questions (5 points), **due date: 23:59 Wednesday, Week 7**
 - Logical formula and predicate logic
 - Z3's knowledge and translation rules
- Lab-Exercise-2 (5 points), **due date: 23:59 Wednesday, Week 7**
 - **Goal:** Manually translate code into z3 formulas/constraints and verify the assertions embedded in the code.
 - **Specification:**<https://github.com/SVF-tools/Software-Security-Analysis/wiki/Lab-Exercise-2>
 - **SVF Z3 APIs:** <https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVF-Z3-API>
- Assignment-2 (25 points) **due date: 23:59 Wednesday, Week 8**
 - **Goal:** automatically perform assertion-based verification for code using static symbolic execution.
 - **Specification:**<https://github.com/SVF-tools/Software-Security-Analysis/wiki/Assignment-2>

Hints for Lab-Exercise-2 Cases

- Lab-Exercise-2 Validation Code
 - `test0()` has an example for validating your translation. Given a program `prog` and an assert `Q`, you are expected to (1) use `checkNegateAssert` to translate the negation of `Q` and check **unsat** of $prog \wedge \neg Q$; `checkNegateAssert` returns **true** means the non-existence of counterexamples, and (2) also **evaluate individual variables' values** (e.g., `a`) if you know `a`'s value is 3. For example, `z3Mgr->getEvalExpr(getZ3Expr("a")) == getZ3Expr(3)`.
 - Closed-world programs, checking **sat** of $prog \wedge Q \equiv$ checking **unsat** $prog \wedge \neg Q$
- `addToSolver(e1)` vs `getEvalExpr(e2)`
 - `e1` is added as a constraint to the solver, while `e2` is not added to the solver hence its truth depends on a particular model (one solution).
- Memory allocations: `p = &a`;
 - `a` is address-taken by `p`, hence an object `&a` needs to be created via `a_addr = getMemObjAddress("&a");`
- Interprocedural (call and return)
 - Bookkeeping the calling context to distinguish local variables.

Methods to Be Implemented

You need to implement the following six functions in Assignment-2:

- `SSE::reachability`
- `SSE::collectAndTranslatePath`
- `SSE::handleCall`
- `SSE::handleRet`
- `SSE::handleNonBranch`
- `SSE::handleBranch`
- The required implementation parts are indicated with TODO comments, and you only need to fill up the code template if a method is partially implemented.